LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING

Daily 2026-10-26 PHP2Shell CVE-2026-109618 and CVE-2026-109619 Active Exploitation Priority Update

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-10-26
Version	1.0
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

Published: 2026-10-26 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
8. Mitigation and Workarounds
9. Detection Engineering
10. Threat Hunting
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. References

Daily 2026-10-26 PHP2Shell CVE-2026-109618 and CVE-2026-109619 Active Exploitation Priority Update

1. Executive Summary

PHP2Shell refers to CVE-2026-109618 and CVE-2026-109619, two critical PHP vulnerabilities reported as capable of remote code execution in default PHP configurations. Datadog Security Labs identifies both CVEs and urges rapid updates; reporting citing Amazon Threat Intelligence states an attempted exploitation event was observed on 17 October, before the 21 October emergency patches. This combination makes internet-facing PHP estates a priority for patching and retrospective web-tier hunting.

The public reporting reviewed supports urgent exposure reduction and investigation. It does not establish a complete victim list, actor attribution, or a universal request signature. Patch using the PHP project/vendor package guidance applicable to the deployed distribution, inventory all PHP runtimes including embedded/control-panel stacks, and inspect the pre-patch period for web-shell, process, outbound-network, and credential-access signals.

2. Vulnerability Metadata

Field	Value
CVEs	CVE-2026-109618; CVE-2026-109619
Family label	PHP2Shell (industry/research label)
Product	PHP runtime; deployment-specific exposure must be validated
Reported impact	Remote code execution in default PHP configurations
Patch event	Emergency releases reported 21 October 2026
Exploitation evidence	Amazon Threat Intelligence reporting cites an observed attempt on 17 October
Confidence	Medium: claims are based on security research and secondary reporting, not a vendor advisory retrieved in this run

3. Source Review & Confidence

Source	Contribution	Confidence
Datadog Security Labs	CVE IDs, severity, affected runtime risk and update urgency	High
BleepingComputer citing Amazon Threat Intelligence	Pre-disclosure exploitation timing and operational context	Medium-High
The Hacker News citing Amazon telemetry	Independent supporting account of observed attempt timing	Medium

4. Vulnerability Details

The risk is amplified by PHP’s broad deployment footprint: internet-facing applications, administrative panels, CMS stacks, APIs, build systems, and packaged appliances may ship distinct PHP versions. A host can appear patched while another runtime, container image, FPM pool, or vendor-supplied component remains exposed. Treat version inventory and service ownership as detection inputs, not merely patch-management fields.

5. Attack Scenarios

- An exposed PHP application receives crafted input targeting a vulnerable runtime, followed by command execution or web-shell placement.
- A compromised web application worker launches an interpreter, shell, downloader, or outbound command-and-control connection.
- A remediation only updates the OS package while containers, FPM pools, control-panel runtimes, or immutable images continue serving vulnerable PHP.

6. Threat Landscape & Exploitation Status

The Amazon-telemetry reporting makes this an exploitation-informed priority. It is not proof of intrusion at any specific organization. Analysts should avoid attributing every PHP web-shell or failed request to these CVEs; correlate application logs, runtime version, execution lineage, and change records.

7. Affected Products and Exposure

Exposure pattern	Priority	Defender action
Public PHP application/API	Critical	Identify runtime/version, update, and inspect access/process logs
Containerized PHP/FPM	Critical	Rebuild images, redeploy, and verify running image digests
CMS/control panel with bundled PHP	Critical	Confirm vendor package guidance and each bundled runtime
Internal-only PHP service	High	Patch on expedited schedule and restrict lateral access

8. Mitigation and Workarounds

1. Enumerate PHP versions across hosts, containers, FPM pools, control panels, CI systems, and appliance components.
2. Apply the PHP project’s fixed release or the supported vendor package update; record both package and effective runtime version.
3. Rebuild and redeploy affected images; do not rely only on a package update in a build host.
4. Reduce public exposure with WAF/reverse-proxy controls while patching, without representing temporary filtering as a fix.
5. Preserve web, PHP-FPM, audit, EDR, and outbound DNS/network logs covering at least the pre-patch window.
6. Rotate application secrets where investigation finds suspicious code execution or web-root modification.

9. Detection Engineering

Signal	Telemetry	Analytic goal
PHP worker spawning shell/interpreter	EDR process telemetry	Detect likely post-exploitation execution
New/changed executable PHP files	FIM, EDR, web-root audit	Detect web-shell or payload placement
PHP worker outbound network traffic	Network/EDR/DNS	Identify retrieval or C2 behavior

```
DeviceProcessEvents
| where Timestamp > ago(30d)
| where InitiatingProcessFileName in~ ("php", "php-fpm", "apache2", "httpd", "nginx")
| where FileName in~ ("sh", "bash", "dash", "cmd.exe", "powershell.exe", "curl", "wget", "python",
```

```
"perl")
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName, FileName, ProcessCommandLine
| order by Timestamp desc
```

10. Threat Hunting

Hypothesis: Exploitation of a vulnerable PHP runtime will often be followed by suspicious child-process execution, web-root modification, or anomalous outbound connections from the web-service identity.

```
DeviceFileEvents
| where Timestamp > ago(30d)
| where FolderPath has_any ("/var/www", "/srv/www", "inetpub", "htdocs")
| where FileName endswith ".php"
| where ActionType in~ ("FileCreated", "FileModified", "FileRenamed")
| project Timestamp, DeviceName, FolderPath, FileName, InitiatingProcessFileName,
InitiatingProcessCommandLine
| order by Timestamp desc
```

Pair hits with web-server logs, deployed release history, and application ownership. This query identifies investigation candidates, not CVE-specific proof.

11. MITRE ATT&CK Mapping

Technique	Name	Rationale
T1190	Exploit Public-Facing Application	Initial-access hypothesis for exposed PHP apps
T1059	Command and Scripting Interpreter	Expected follow-on shell/interpreter execution
T1505.003	Web Shell	Hunt focus for malicious server-side PHP files

12. Validation / Lab Testing

No exploit was executed. This report used source corroboration and analytic review only. Validate updates against the authoritative PHP/vendor release notes and perform approved application regression testing before broad production rollout.

13. Recommended Actions Summary

Priority	Owner	Action
Critical	App/platform owners	Inventory and patch all externally reachable PHP runtimes
Critical	Cloud/container owners	Rebuild and redeploy PHP images and verify running versions
High	SOC	Hunt for PHP-child process, web-root, and egress anomalies
High	Application security	Confirm WAF controls and secret-rotation procedures

14. References

See `sources.md` and `references.md` for URLs, evidence roles, and limitations.